

1. Method Overriding

2. final Variable

3. final Method

4. final Class

5. Abstract Method

1. Method Overriding

☞ When a **child class provides its own version** of a method that is already defined in the **parent class**.

- Method name, return type, and parameters must be **same**.
- Used for **runtime polymorphism**.

Example:

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

```
public class Test {
```

```
public static void main(String[] args) {  
    Animal a = new Dog(); // parent reference, child object  
    a.sound(); // Output: Dog barks  
}  
}
```

2. final Variable

☞ A variable declared with **final** cannot be **changed** (becomes constant).

- Must be initialized only **once**.

Example:

```
class Test {  
    final int speed = 100;  
    void show() {  
        // speed = 200; // ✗ Error, cannot change final variable  
        System.out.println("Speed = " + speed);  
    }  
}
```

3. final Method

☞ A method declared with **final** cannot be **overridden** by child classes.

Example:

```
class Parent {  
    final void display() {  
        System.out.println("This is a final method");  
    }  
}
```

```
class Child extends Parent {
```

```
// void display() { } // ✗ Error: Cannot override final method  
}
```

4. final Class

☞ A class declared as **final** cannot be **inherited**.

Example:

```
final class Vehicle {  
    void run() {  
        System.out.println("Running...");  
    }  
}
```

```
// class Car extends Vehicle { } // ✗ Error: Cannot inherit from final class
```

5. Abstract Method

☞ A method that is declared but **not implemented** (no body).

☞ It ends with a **semicolon (;)**.

☞ Must be inside an **abstract class**.

Example:

```
abstract class Shape {  
    abstract void draw(); // abstract method (no body)  
}
```

```
class Circle extends Shape {  
    void draw() {  
        System.out.println("Drawing Circle");  
    }  
}
```

}

6. Abstract Class

☞ A class declared with **abstract** keyword.

- Can have **abstract** methods (without body).
- Can also have **normal methods**.
- Cannot be **instantiated** (we cannot create an object directly).

Example:

```
abstract class Shape {  
    abstract void draw(); // abstract method  
  
    void show() { // normal method  
        System.out.println("This is a shape");  
    }  
}  
  
class Rectangle extends Shape {  
    void draw() {  
        System.out.println("Drawing Rectangle");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Shape s = new Rectangle(); // upcasting  
        s.draw(); // Output: Drawing Rectangle  
        s.show(); // Output: This is a shape  
    }  
}
```

✓ Quick Summary Table

Keyword / Concept	Meaning
Method Overriding	Child class redefines parent method
<code>final</code> variable	Constant (cannot be changed)
<code>final</code> method	Cannot be overridden
<code>final</code> class	Cannot be inherited
Abstract method	Declared, no body, must be in abstract class
Abstract class	Cannot be instantiated, may contain abstract + normal methods